Smart Crop Disease Detection and Yield Prediction System: Project Report

Oct 29, 2026 · @SD

Abstract

Farmers lose crops to leaf diseases that are spotted late, and they plan without a quick estimate of what a field is likely to produce. This project builds one offline web application that does both. For disease detection, each leaf photo is turned into 399 colour, texture and edge measurements and classified by a gradient boosting model into 38 crop-disease classes, trained on 11,227 PlantVillage images. For yield, a gradient boosting regressor trained on 19,689 Indian state-level records (1997–2020) estimates yield per hectare from crop, state, season, year, rainfall, fertiliser and pesticide, and scales it by field area. On held-out data the disease model reached 92.3% accuracy and the yield model a typical error of about 16% when predicting 2015–2020 from earlier years. The app answers "Not sure" below 50% confidence and rejects photos that are clearly not leaves. These figures come from lab-style images and state-level data, so they are upper bounds: the disease model is classical machine learning, covers 14 crops (not rice or wheat), and has not been tested on field photos or in a field trial.

Keywords: crop disease detection, yield prediction, PlantVillage, gradient boosting, Flask, precision agriculture

1. Introduction

This project builds a web application that does two jobs for a farmer or an agriculture student: it identifies a crop disease from a leaf photo, and it estimates the yield a field is likely to give.

1.1 Background and motivation

Crop diseases are usually spotted by eye, and the person looking needs training. A wrong or late call means the wrong spray, or no spray until the disease has spread. Yield estimates have a similar problem: farmers and planners reason from experience, while records of past yields by crop, state and season already exist in public data. Machine learning can read both kinds of evidence, photos and records, and turn them into an answer in seconds.

1.2 Problem statement

Build a low-cost, offline-capable web system that (a) classifies a leaf image into a crop and disease class, with an honest confidence score, and (b) predicts yield per hectare and total production from crop, state, season, year, rainfall, fertiliser and pesticide use.

1.3 Objectives

1. Collect and prepare public datasets for leaf disease images and for Indian crop yields.
2. Train and compare several models for each task, and measure them on data the models have not seen.
3. Return a confidence score with every disease result, and refuse to answer when confidence is low.
4. Give practical, general crop-protection steps for each detected disease.
5. Package everything as a web app that runs on one laptop with no internet connection.
6. Document the limits of the system openly.

1.4 Scope

In scope: 38 crop and disease classes across 14 crops (PlantVillage); yield for 55 crops across 30 Indian states and union territories; a browser interface; a JSON API; automated tests.

Out of scope: diagnosis from field photos with complex backgrounds, soil analysis, live weather feeds, satellite imagery, and a link between detected disease and yield loss. The two models are independent.

2. Literature Survey

Published work shows that image-based disease detection reaches very high accuracy on lab-style photos but drops sharply in the field, and that yield prediction mostly relies on weather, soil and input data. The studies below shaped the design choices in this project. Numbers in square brackets refer to the References.

2.1 Image-based plant disease detection

The PlantVillage project released a large open collection of labelled leaf photos in order to support mobile disease diagnosis [1]. Mohanty, Hughes and Salathé used 54,306 of these images, covering 14 crops and 38 crop-disease classes, to train AlexNet and GoogLeNet from scratch and by transfer learning. Their best model, GoogLeNet with transfer learning on colour images, scored 99.35% on a held-out test set [2]. The same paper reports the catch: on small sets of online images taken under different conditions, the best models

managed only about 31% top-1 accuracy [2]. Geetharamani and Pandian trained a nine-layer deep convolutional network on a PlantVillage-derived dataset for leaf disease identification [3].

A survey of 40 deep learning studies in agriculture concludes that deep learning generally beats conventional pipelines built on hand-crafted features with classifiers such as SVMs and random forests, because it learns features itself. The same survey warns that studies use different datasets and metrics, so results are hard to compare directly, and that deep learning needs large labelled datasets and longer training [4].

2.2 Crop yield prediction

A systematic review of 50 machine learning studies on yield prediction found that temperature, rainfall and soil type were the most-used features, that artificial neural networks were the most-used machine learning algorithm, and that among deep learning methods CNNs were the most common, followed by LSTM and deep neural networks [5]. A regional example is the functional artificial neural network of Jena et al., which predicted crop yield in Karnataka from 1991 to 2012 using total rainfall, cumulative temperature distribution, share of irrigated land and average fertiliser use, and beat a linear regression baseline [6].

2.3 Datasets used by this project

Task	Dataset	Size	Source
Leaf disease	PlantVillage colour images	54,305 images, 14 crops, 38 classes	[1], [8]
Yield	Indian crop yield by state, 1997–2020 (crop, season, state, area, production, rainfall, fertiliser, pesticide, yield)	19,689 records, 55 crops, 30 states	[7]

2.4 Summary of related work

Study	Task	Method	Reported finding
Mohanty et al., 2016 [2]	Disease classification	AlexNet, GoogLeNet; scratch and transfer learning	99.35% on held-out lab images; about 31% on out-of-condition images

Study	Task	Method	Reported finding
Geetharamani and Pandian, 2019 [3]	Disease classification	Nine-layer deep CNN	Applied to PlantVillage-derived data
Kamilaris and Prenafeta-Boldú, 2018 [4]	Survey of 40 studies	Deep learning versus conventional methods	Deep learning generally more accurate; comparisons are hard
van Klompenburg et al., 2020 [5]	Review of 50 yield studies	Mixed ML and deep learning	Rainfall, temperature and soil are the usual features
Jena et al., 2023 [6]	Yield, Karnataka	Functional ANN	Beat linear regression on RMSE, MSE and MAE

2.5 Research gap and positioning

Three gaps stand out in the works reviewed here:

1. High accuracy is often reported on lab-style test images only, while performance on field-style images is much lower [2]. A usable tool should therefore report its confidence and decline to guess when unsure.
2. Disease detection and yield prediction are normally studied as separate problems, with separate code and no common interface for a farmer.
3. Most published systems are research code. A tool that runs offline on one laptop, with a simple interface, is more useful at a farm gate or an exhibition.

This project addresses these points by (a) adding a confidence threshold with an explicit "Not sure" answer, (b) putting both models behind one web interface and one API, and (c) running fully offline. It does not claim to beat the deep learning results above: the disease model here uses hand-crafted features, and its accuracy is reported against held-out lab-style images only (Section 7).

3. Feasibility Study and Requirements

The project is feasible with free software, public data and an ordinary laptop with no GPU. Both models train in minutes on two CPU cores, which was confirmed during development.

3.1 Feasibility

Area	Assessment
Technical	All needed libraries (scikit-learn, scikit-image, Flask) are mature and run on CPU. Datasets are public and large enough for the task. Deep learning frameworks were not available in the build environment, so classical computer-vision features were used instead.
Economic	No licence cost: every component is open source and every dataset is free to use. No cloud service is needed to run the app.
Operational	The app runs offline in a browser on one machine, uses a plain interface, and needs no account or login. A farmer or student can use it after a one-line explanation.
Data and legal	PlantVillage is distributed openly, and the yield data is published under a CC BY 4.0 licence. Uploaded photos are processed in memory and are not stored.
Schedule	Four working modules (data, models, API, interface) with a clear order of dependency; see Section 5.

3.2 Multidisciplinary aspects

The work combines agronomy (disease symptoms, crop protection), computer vision (colour, texture and edge features), statistics (regression, hold-out evaluation) and web engineering (API design, validation, usability).

3.3 Functional requirements

1. Accept a JPG or PNG leaf photo of up to 6 MB and return the top three crop-disease classes with probabilities.
2. Return "Not sure" when the top probability is under 50%.
3. Show the likely cause (fungal, bacterial, viral, pest) and general protection steps.
4. Accept crop, state, season, year, rainfall, fertiliser per hectare, pesticide per hectare and field area, and return yield per hectare and total production.
5. Compare the estimate with the typical range for that crop in historical records.

6. Reject invalid input with a message that says what to fix.
7. Show measured model performance and known limits inside the app.

3.4 Non-functional requirements

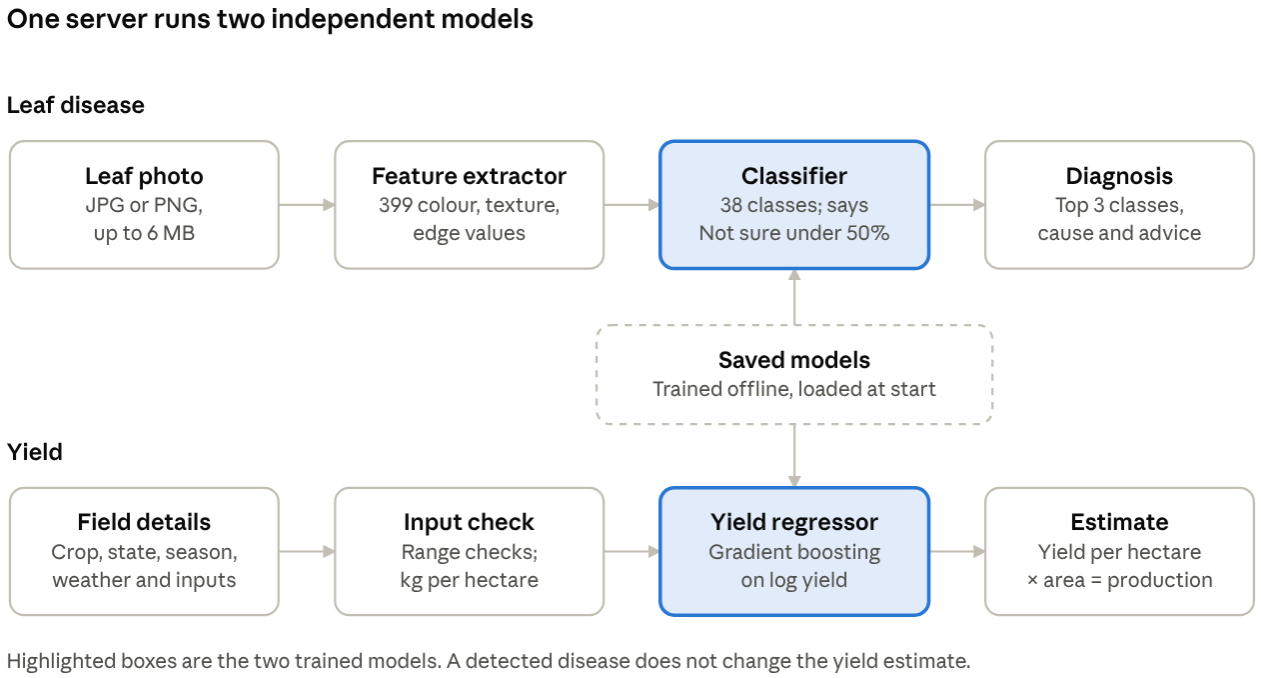
- Runs offline on a laptop with 4 GB of RAM or more and no GPU.
- Answers a leaf photo in well under a second on a standard CPU.
- Works on a phone-sized screen and with keyboard navigation.
- Never stores user photos.

3.5 Bill of materials (software and hardware)

Component	Purpose	Cost
Python 3.13	Language runtime	Free
Flask 3.1	Web server and JSON API	Free
scikit-learn 1.9	Classifiers, regressors, metrics	Free
scikit-image 0.26	Colour, LBP, GLCM and HOG features	Free
pandas, NumPy, Pillow, joblib, Matplotlib	Data handling, images, model storage, plots	Free
PlantVillage images	Disease training data	Free (openly distributed copy)
Indian crop yield records	Yield training data	Free (CC BY 4.0)
Laptop or desktop, 2 CPU cores, 4 GB RAM or more	Training and running the app	Existing machine
Phone or camera (optional)	Taking leaf photos	Existing device

4. Methodology and System Design

The system has two independent models behind one Flask server. A leaf photo goes through a feature extractor and a classifier; the field details go through a regression model. Both are trained offline and loaded when the server starts.



system architecture · 2 pipelines, 1 shared server

The disease pipeline (top) and the yield pipeline (bottom) share only the server and the saved model files.

4.1 Disease detection pipeline

Data. 11,227 PlantVillage images: up to 300 randomly chosen images per class, across all 38 classes (classes with fewer images contribute all they have). The images were fetched from the public GitHub copy of the dataset.

Features (399 numbers per image). Each photo is resized to 128 × 128 and described by:

Group	What it captures	Values
Colour	HSV histograms; mean and spread of RGB, HSV and Lab channels; share of brown, yellow, dark and green pixels	57
Texture	Local binary pattern histogram; five grey-level co-occurrence statistics (contrast, dissimilarity, homogeneity, energy, correlation)	15
Shape and edges	Histogram of oriented gradients on a 64 × 64 grey thumbnail; edge strength statistics	327

Colour matters because lesions change leaf colour; texture and edges capture spots, rings and mould.

Models and protocol. The images are split 70% train, 15% validation and 15% test, stratified by class. Three classifiers are trained on the training split: ExtraTrees (400 trees), an RBF support vector machine on standardised features, and histogram gradient boosting. The one with the best validation accuracy is refitted on train plus validation and scored once on the test split. At prediction time the top probability must reach 50%, otherwise the app answers "Not sure".

4.2 Yield prediction pipeline

Data. 19,689 records of crop, season, state, year, area, production, rainfall, fertiliser, pesticide and yield, for 55 crops and 30 states and union territories, 1997 to 2020.

Preparation.

1. Fertiliser and pesticide in the raw file are state totals in kilograms, so both are divided by area to give kilograms per hectare, which is what a farmer can enter.
2. Yield spans several orders of magnitude (coconut is counted in nuts), so the model predicts the logarithm of yield and the result is converted back.
3. Crop, state and season are used as categories. Year, rainfall, fertiliser per hectare and pesticide per hectare are numeric.
4. Area is not a model input. In this dataset it is the state's total area under the crop, so using it makes the model fail for a single farm. The app multiplies yield per hectare by the farmer's own area to get production.

Models and protocol. A ridge regression and a random forest serve as baselines against histogram gradient boosting. Two tests are run: a random 80/20 split, and a temporal hold-out that trains on 1997–2014 and tests on 2015–2020, which is the honest check for predicting a future year.

4.3 Key design decisions

Decision	Reason
Hand-crafted image features with a scikit-learn classifier	Deep learning frameworks could not be installed in the build environment; the approach also trains in minutes on CPU and is easy to explain in a viva
50% confidence threshold with a "Not sure" answer	A wrong confident answer is worse than asking for a better photo
Test set used once, after model selection on validation	Avoids choosing the model that happens to fit the test data

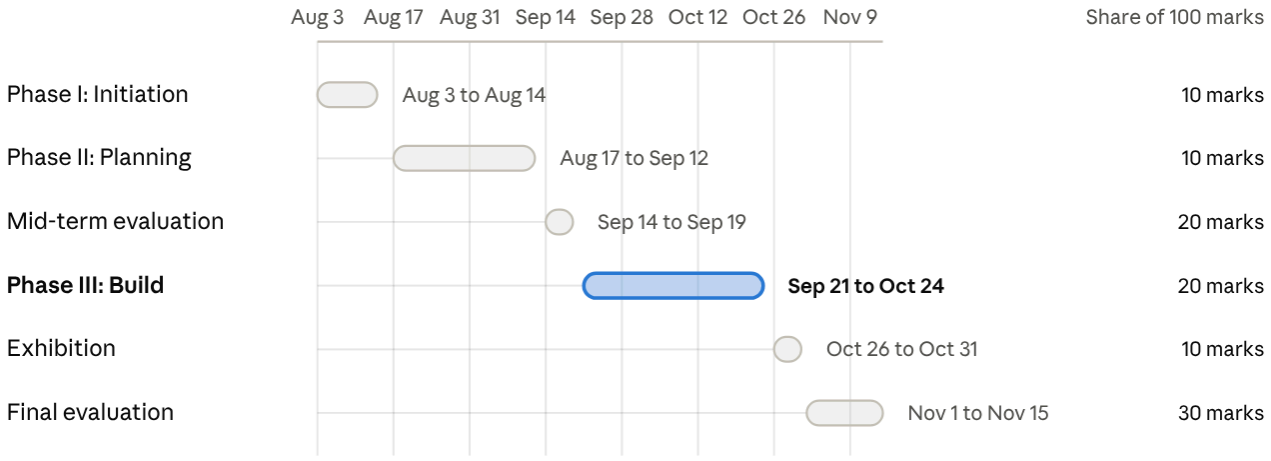
Decision	Reason
Temporal hold-out for yield	A random split lets the model see the same state and crop in later years
Per-hectare inputs, area only scales production	Keeps the model valid for a single farm, and fixed a real failure found in testing (Section 7.5)
Photos processed in memory only	No user data is stored

5. Project Planning

The institute schedule has six assessed stages between 3 August and mid-November 2026. Implementation is the longest, at 34 days, and the project has to deliver a working system by its end.

Implementation is the longest phase, 21 Sept to 24 Oct 2026

Institute schedule for the major project, 2026



Final evaluation = report 10, presentation and viva 15, paper or patent 5. It ends in mid-November; drawn to 15 Nov.

major project evaluation rubric, 2026-27 · 6 stages

5.1 Work breakdown structure

Work package	Tasks	Phase	Output
1. Problem and survey	Problem definition, literature survey, objectives, feasibility	I	Sections 1 to 3 of this report

Work package	Tasks	Phase	Output
2. Design	Architecture, methodology, risk list	II	Section 4
3. Data	Choose datasets, fetch the image subset, clean yield records, define features	II to III	`data/`, `core/features.py`
4. Disease model	Feature extractor, train three classifiers, evaluate, confusion matrix	III	`models/disease_model.joblib`, `reports/`
5. Yield model	Per-hectare features, baselines, temporal hold-out	III	`models/yield_model.joblib`
6. Web application	Flask API, interface, input validation	III	`app/`
7. Testing	Automated API tests, interface check, defect fixes	III	`tests/`, Section 7
8. Documentation	README, report, poster content	Exhibition, final	`README.md`, this report

5.2 Risks and fallbacks

Risk	Effect	Mitigation
Deep learning frameworks not installable in the build environment	Lower ceiling on image accuracy	Hand-crafted features with tree and SVM models; limit stated openly; MobileNet transfer learning kept as the upgrade path
Training photos are lab-style	Weak accuracy on field photos	50% confidence rule and a "Not sure" answer with retake advice
Same physical leaf in train and test	Inflated accuracy figure	Disclosed; the figure is treated as an upper bound

Risk	Effect	Mitigation
Yield data is state-level	Estimates describe typical outcomes, not one farm	Stated in the app; typical range shown beside each estimate
Dataset licence	Reuse restrictions	Only openly licensed sources, cited in the References
Short build window	Missed demo	Scope kept to two models and one interface, with automated tests

6. Implementation

The system is a small Flask application with a plain HTML, CSS and JavaScript front end. It uses no external fonts, scripts or services, so it runs with the network switched off.

6.1 Code organisation

Path	Responsibility
`app/app.py`	Flask server: page, JSON API, input validation, model loading
`app/templates/`, `app/static/`	The interface: three tabs, drop zone, result labels, results page
`core/features.py`	The 399-value leaf feature extractor, shared by training and the server
`core/advice.py`	General crop-protection guidance per disease class
`scripts/train_disease.py`	Feature extraction, model comparison, test scoring, confusion matrix
`scripts/train_yield.py`	Yield cleaning, baselines, temporal hold-out, feature importance, saved model
`scripts/fetch_images.py`, `fetch_all.sh`	Helpers that pull the image subset from a public GitHub copy of PlantVillage
`models/`, `reports/`	Saved models, metrics in JSON, confusion matrix image

Path	Responsibility
`tests/test_app.py`	Automated tests of the API

The feature extractor lives in one file used by both training and the server, so the two can never drift apart.

6.2 API

Method and path	Input	Output
`POST /api/disease`	Multipart form with `image` (JPG or PNG, up to 6 MB)	Top class with crop, condition, cause, summary and steps; top three classes with probabilities; a flag for whether confidence reached 50%
`POST /api/yield`	JSON: crop, state, season, year, area (ha), rainfall (mm), fertiliser and pesticide (kg per ha)	Yield per hectare, production, unit, typical range for the crop, and notes
`GET /api/meta`	None	Crops, states, seasons, per-crop defaults, which states grow which crop
`GET /api/report`	None	Training and test metrics for both models
`GET /health`	None	Whether each model is loaded

6.3 Interface

- **Check a leaf.** Drag a photo in or pick a file. The result appears as a specimen-style label: crop, condition, cause, a confidence ruler with the 50% line marked, what the disease looks like, what to do, and the other two likeliest classes. Below 50% the label says "Not sure" and asks for a better photo.
- **Estimate yield.** Choosing a crop fills typical fertiliser, pesticide and rainfall values that the user can change. The result shows production, yield per hectare, and a band marking the middle 80% of historical yields for that crop, with a pin for this estimate. Warnings appear for crop-and-state pairs absent from the training data and for years beyond 2020.
- **How well it works.** Measured accuracy, the model comparison, the confidence trade-off table, the confusion matrix and the list of limits.

The page is responsive down to phone width, has visible keyboard focus, and all server text is inserted as plain text, never as HTML.

6.4 Validation and privacy

Check	Behaviour
Upload size	Over 6 MB returns HTTP 413 with a plain message
File type	Only `.jpg`, `.jpeg`, `.png`; the file is also opened with an image library, so a renamed text file is rejected
Categories	Crop, state and season must be values the model knows
Numbers	Year 1990 to 2050; area 0.01 to 100,000 ha; rainfall 50 to 7,000 mm; fertiliser 0 to 2,000 kg per ha; pesticide 0 to 50 kg per ha
Photo check	Images with too little or too much edge detail, or almost no plant colour, return HTTP 422 with a reason and a retake hint; thresholds are learned from the training images
Photo storage	None. Images are read into memory, converted to features, and discarded

7. Testing and Results

7.1 Test plan

Testing had four parts. (1) Twelve automated API tests (pytest, all passing) cover health, metadata, valid and invalid disease uploads, the photo check, valid and invalid yield requests and an end-to-end run on a real sample leaf. (2) Held-out model evaluation: the leaf data was split 70/15/15 (train/validation/test, stratified); models were chosen on validation and the test split was scored once. (3) An end-to-end run of 144 leaf images that were never used in training, sent through the real app. (4) A manual check of the interface at desktop and phone widths.

7.2 Disease detection results

The data set had 11,227 images over 38 classes (7,858 train, 1,684 validation, 1,685 test).

Candidate model	Validation accuracy
ExtraTrees (400 trees)	87.3%
SVM (RBF)	89.9%
HistGradientBoosting (chosen)	91.9%

On the untouched test split the chosen model reached **92.3% accuracy and 92.3% macro-F1**. On the 144 unseen images run through the live app, 91.7% were correct. Median prediction time was 73 ms.

The app answers "Not sure" when the top probability is under 50%. Raising the threshold trades coverage for accuracy:

Confidence threshold	Share of photos answered	Accuracy when answered
0.3	99.9%	92.4%
0.5 (used in the app)	98.0%	93.4%
0.7	94.5%	94.7%
0.9	88.6%	96.9%

The weakest classes (F1) were tomato early blight (0.79), tomato late blight (0.79), corn grey leaf spot (0.81), tomato Septoria leaf spot (0.83) and tomato yellow leaf curl virus (0.86). These diseases produce similar brown spots, which hand-made colour and texture features separate less well. Healthy leaves of cherry, corn and strawberry scored 0.99.

These figures are an upper bound. PlantVillage photos are lab-style and several images can come from the same physical leaf, so a random split may place near-copies in both train and test. Field photos will score lower.

7.3 Yield prediction results

Evaluation	Model	R^2 (yield)	MAE	Median error
Random 80/20	Ridge baseline	0.766	34.2	34.2%
Random 80/20	Random forest	0.965	10.7	12.1%
Random 80/20	Gradient boosting (chosen)	0.880	16.1	14.0%

Evaluation	Model	R^2 (yield)	MAE	Median error
Temporal (train to 2014, test 2015–2020)	Gradient boosting (chosen)	0.890	19.8	16.4%
Temporal	Random forest	0.973	9.7	15.9%

MAE is in the data set's yield units (mostly tonnes per hectare; coconut is nuts per hectare).

Honest note on model choice. Gradient boosting was chosen because it is small (2.4 MB against 67.5 MB for the forest), trains in seconds and fits the log of yield as well as the forest (R^2 0.938 against 0.937 on the temporal test). The random forest, however, scored better on raw-yield R^2 and mean error in both tests, while its typical (median) error is about the same (15.9% against 16.4%). The raw-yield gap is probably driven by a few very high-yield crops such as coconut and sugarcane. A forest would be a reasonable alternative if file size is not a concern. The system quotes a typical error of about 16%, not a precise figure.

Permutation importance shows that the crop dominates (1.68), followed by state (0.23). Season, year and rainfall each add little (0.01–0.02), and fertiliser and pesticide almost nothing (0.003 and 0.002). The model therefore mostly learns what a crop typically yields in a state; it should not be used to judge the effect of changing inputs.

7.4 Photo check results

The photo check rejected 27 of 11,227 training leaves (0.24%) and none of the 144 unseen leaves. Of 19 non-leaf test inputs, 15 were rejected. Four colourful photographs (an astronaut, a coffee cup, a cat and a rocket) passed, so the check cannot be relied on to catch every non-leaf image.

7.5 Defects found and fixed

Defect	How found	Fix
Yield model used state-wide area as an input, giving about 1.4 t/ha for wheat in Punjab	Manual sanity check of outputs	Area removed from the model; used only to scale production
Non-leaf images got a label with 100% confidence	Trial with a plain green image	Photo check added, with a test
Page overflowed by 26 px on a phone	Phone-width check	Tables wrapped in a scrolling container

Defect	How found	Fix
Stale text about area and an unverified licence claim	Review	Text corrected

8. Limitations and Risks

The reported accuracy applies to clean, lab-style leaf photos and to typical crop-level outcomes, not to every field and every farm. The main limits are listed here so that a reader can judge how far to trust each result.

- **Lab-style photos.** PlantVillage images show one leaf on a plain background. Published work on this dataset found accuracy fell to about 31% on images taken under other conditions [2]. The same drop should be expected here. The 50% confidence rule reduces, but does not remove, confident wrong answers.
- **Possible leakage between train and test.** More than one image can come from the same physical leaf, and a random split can place them on both sides. The disease accuracy figure should be read as an upper bound. A leaf-grouped split is the proper fix.
- **Classical features, not a deep network.** Hand-crafted colour, texture and edge features have a lower ceiling than convolutional networks [4]. Deep learning frameworks could not be installed in the build environment, so this was a constraint, not a preference.
- **Disease coverage.** The disease model knows 14 crops and 38 classes. It cannot recognise rice, wheat, sugarcane or other crops that are in the yield data but not in PlantVillage, and it will force an unfamiliar leaf into one of its 38 classes unless confidence is low. The photo check rejects blank, noisy and many non-leaf images (15 of 19 tested), but four colourful non-leaf photographs passed it, so it is a safety net, not a leaf detector.
- **Yield describes typical outcomes.** The data is state-level, so the model captures what a crop usually yields in a state and season. It has no soil, variety, irrigation or in-season weather. Rainfall, fertiliser and pesticide add little beyond crop and state (Section 7.3).
- **Years beyond the data.** Records end in 2020. Later years are extrapolated, and the app says so.
- **Unit differences.** Coconut yield is recorded in nuts per hectare, not tonnes.
- **Independent models.** A detected disease does not reduce the yield estimate. Linking disease severity to yield loss needs field data this project does not have.

- **General advice only.** Crop-protection steps are generic. Product choice, dose and legal limits vary by crop, state and season, so users are told to confirm with a local agriculture office.
- **Data provenance.** The yield file was obtained from a public GitHub copy whose columns, row count and year range match the published dataset description [7]. It was not downloaded from the publisher directly.
- **No field trial.** The system has not been tested with farmers or on photos taken in farms.

9. Conclusion and Future Work

9.1 Conclusion

The project delivered a working offline web application that does two jobs: it identifies a crop disease from a leaf photo and it estimates the yield of a field. The disease model recognises 38 crop-disease classes with 92.3% test accuracy and says "Not sure" when its confidence is under 50%. The yield model predicts later years with a typical error of about 16%, against 34% for a simple baseline. Both sit behind one Flask API and one interface, with input checks, a photo check and twelve automated tests, and all measured results are reported with their limits.

The work also showed where simple results mislead. A first yield model that used area looked good but gave impossible answers for a single farm; a plain green image was once labelled with full confidence. Both were found by testing outputs rather than metrics, and both were fixed.

9.2 Future work

1. Replace the hand-made features with a small deep network (for example MobileNet with transfer learning) once a deep-learning environment is available, and measure the gain on the same split.
2. Collect and test on real field photos, with different light and backgrounds, and split by physical leaf to remove the same-leaf leakage.
3. Strengthen the photo check, for example with a learned leaf / not-leaf classifier, since four colourful non-leaf photos still pass.
4. Add rice and wheat leaf diseases, which the current data does not cover.
5. Move from state-level to district- or farm-level yield data, and add soil and in-season weather inputs.
6. Run a small field trial with farmers or the agriculture office and record how often the advice is useful.

References

1. D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015.
2. S. P. Mohanty, D. P. Hughes and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, article 1419, 2016. doi:10.3389/fpls.2016.01419
3. G. Geetharamani and A. Pandian, "Identification of plant leaf diseases using a nine-layer deep convolutional neural network," Computers & Electrical Engineering, vol. 76, pp. 323–338, 2019.
4. A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018.
5. T. van Klompenburg, A. Kassahun and C. Catal, "Crop yield prediction using machine learning: A systematic literature review," Computers and Electronics in Agriculture, vol. 177, article 105709, 2020.
6. P. R. Jena, B. Majhi, R. Kalli et al., "Prediction of crop yield using climate variables in the southwestern province of India: a functional artificial neural network modeling (FLANN) approach," Environment, Development and Sustainability, vol. 25, pp. 11033–11056, 2023. doi:10.1007/s10668-022-02517-x
7. Indian state-level crop yield data set, 1997–2020 (crop, season, state, area, production, rainfall, fertiliser, pesticide, yield), Mendeley Data, doi:10.17632/ncw2vbcgnk. The copy used was downloaded from a public GitHub repository and checked against the description of this data set.
8. PlantVillage-Dataset, public GitHub copy of the PlantVillage images, github.com/spMohanty/PlantVillage-Dataset.
9. Major project evaluation rubric, 2026–27 (institute document supplied with the project brief).

Appendix

A. How to run

```
pip install flask scikit-learn scikit-image pandas numpy pillow joblib
matplotlib
python app/app.py          # open http://127.0.0.1:5000
python -m pytest tests     # 12 tests
```

To retrain: `python scripts/train_yield.py`, `python scripts/train_disease.py data/images`, then `python scripts/train_gate.py` for the photo check. `python scripts/yield_rf_check.py` repeats the random forest versus gradient boosting

comparison on the temporal split.

B. Example API call

```
curl -X POST http://127.0.0.1:5000/api/yield -H "Content-Type:
application/json" -d
'{"Crop":"Rice","State":"Punjab","Season":"Kharif","Crop_Year":2020,"Area":1,
"Annual_Rainfall":1000,"Fert_per_ha":120,"Pest_per_ha":0.5}'

curl -X POST http://127.0.0.1:5000/api/disease -F "image=@leaf.jpg"
```

The yield call returns yield per hectare, total production, the unit, and the typical 10th, 50th and 90th percentile for that crop. The disease call returns the top three classes with confidence, a cause and action advice, or a "Not sure" answer below 50%. A photo that fails the photo check returns HTTP 422 with a reason. The exact field names are listed on the `/api/meta` endpoint and in Section 6.2.

C. The 38 classes by crop

Apple (scab, black rot, cedar apple rust, healthy); blueberry (healthy); cherry (powdery mildew, healthy); corn (grey leaf spot, common rust, northern leaf blight, healthy); grape (black rot, esca, leaf blight, healthy); orange (citrus greening); peach (bacterial spot, healthy); bell pepper (bacterial spot, healthy); potato (early blight, late blight, healthy); raspberry (healthy); soybean (healthy); squash (powdery mildew); strawberry (leaf scorch, healthy); tomato (bacterial spot, early blight, late blight, leaf mould, Septoria leaf spot, spider mites, target spot, yellow leaf curl virus, mosaic virus, healthy).